



廣東工業大學

本科毕业设计（论文）

基于多模型集成的轻量化 AI 对话系统设计与实现

学 院	先进制造学院
专 业	人工智能
年级班级	2022 级（3）班
学 号	3122009224
学生姓名	施乔
指导教师	侯艳

2026 年 4 月

摘要

近年来，大语言模型在文本生成和知识问答领域取得了显著进展，带动了一批 AI 对话产品的落地。不过，现有的主流方案几乎都跑在云端，用户的数据不可避免地要经过远程服务器，一方面存在隐私泄露的风险，另一方面持续的算力开销也让个人研究者和小团队望而却步。针对这些痛点，本文设计并实现了 RETALK 系统，它是一个专门面向学术研究场景的轻量化 AI 对话工具。前端这边用的是 Vite + React + TypeScript 的组合，界面样式交给 Tailwind CSS；后端搭在 FastAPI 上面，通过 Ollama 把 llama3.2 这类开源模型跑在本地机器上，完全不依赖外网。系统允许用户上传 PDF 和 DOCX 格式的文档，后端提取出文本之后，利用递归分块的方法把长文本拆成大约 500 个 token 一段，存进 SQLite 数据库。当用户提问时，系统先从库里捞出最相关的几段文本，拼成上下文，再喂给本地模型做 RAG 推理，生成的回答里会用 [1][2] 这样的编号标出引用来源，还能一键导出成 BibTeX 或者 GB/T 7714 格式。用户认证方面采用 JWT 令牌加 bcrypt 密码哈希的方案，保障账户安全。实测下来，API 接口的响应时间控制在 100 毫秒以内，模型推理 30 秒内就能跑完，在一台普通配置的电脑上也能稳定运行。该系统为个人研究者和小团队提供了一种兼顾隐私保护与使用成本的本地化学术对话解决方案。

关键词：AI 对话系统，大语言模型，RAG，轻量化部署，文档解析

Abstract

Large Language Models have recently driven rapid growth in text generation and knowledge-based Q&A, and many AI chatbot products have appeared as a result. Yet almost all mainstream offerings depend on cloud servers, so user data must travel through remote infrastructure, raising privacy concerns. The recurring compute expense also puts these tools out of reach for solo researchers and small groups. This thesis presents RETALK, a lightweight, locally deployed AI dialogue system designed for academic use. On the front end, the project combines Vite, React, and TypeScript; Tailwind CSS handles interface styling. The back end runs on FastAPI and calls open-source models like llama3.2 through Ollama, with no need for an Internet connection. Users can upload PDF or DOCX files. Once the text is extracted, a recursive chunking method splits long content into roughly 500-token pieces stored in SQLite. When a question comes in, the system pulls the most relevant chunks, stitches them into context, and sends the assembled prompt to the local model for retrieval-augmented generation. Each reply carries numbered reference markers like [1] and [2]; users may export these citations to BibTeX or GB/T 7714 format in one step. Account security relies on JWT tokens paired with bcrypt password hashing. In practice, API calls return within 100 ms, model inference finishes inside 30 seconds, and the whole stack runs steadily on a consumer-grade machine. RETALK thus offers individual researchers and small teams a locally hosted academic dialogue tool that keeps data private while controlling costs.

Key words : AI Conversation System, Large Language Model, RAG, Lightweight Deployment, Document Parsing

目录

1 绪论	1
1.1 研究背景及意义	1
1.2 国内研究状况及分析	2
1.3 研究目标与内容	2
1.4 主要技术路线	3
1.4.1 技术选型	3
1.4.2 开发环境	4
2 需求分析	5
2.1 功能需求	5
2.1.1 AI 对话功能	5
2.1.2 文档管理功能	5
2.1.3 会话管理功能	5
2.1.4 引用溯源功能	6
2.2 性能需求	6
2.2.1 服务端性能需求	6
2.3 其他需求	6
2.3.1 健壮性需求	6
2.3.2 易用性需求	7
2.3.3 可扩展性需求	7
2.4 功能模型	7
2.4.1 用例图	7
2.4.2 用例描述	8
2.5 静态模型	12
3 概要设计	14
3.1 系统体系架构设计	14
3.1.1 整体架构	14

3.1.2 后台服务端架构设计	15
3.1.3 前端架构设计	15
3.2 数据设计	16
4 详细设计	17
4.1 系统层次结构	17
4.2 消息发送模块 IPO 表.....	17
4.3 数据库设计	21
5 系统实现	26
5.1 文档管理功能实现	26
5.1.1 界面设计	26
5.1.2 功能实现	27
5.2 对话功能实现	28
5.2.1 界面设计	28
5.2.2 功能实现	30
5.3 用户模块功能实现	32
5.3.1 界面设计	33
5.3.2 功能实现	33
总结与展望	36
参考文献	37
致谢	38

1 绪论

1.1 研究背景及意义

深度学习技术的发展，让大型语言模型（Large Language Model, LLM）在自然语言处理方面有了很大进步^[1,2]。从 GPT-3 到 GPT-4，从 Claude 到 Gemini，这些模型的语言理解和生成能力越来越强。它们在对话、文本创作、代码生成等任务上表现都不错^[3,4,5]。

当前 AI 对话系统存在的问题：

高度依赖云服务：现在主流的 AI 对话产品，比如 ChatGPT、文心一言，都是部署在云端的。用户必须联网，通过网络请求远程服务器才能拿到回复。这有两个问题：一是隐私数据可能泄露，二是网络延迟会影响体验。

部署成本高昂：跑大语言模型需要高性能 GPU。对个人用户和小型团队来说，硬件和运维成本都比较贵。

功能场景局限：很多 AI 对话产品只能做通用聊天。科研人员想用它来读文献、查资料、追溯引用，这些功能大多没有。

交互体验欠佳：很多开源 AI 对话项目只注重功能，界面设计比较粗糙，用起来不太顺手。

理论意义：

探索大语言模型本地部署的可行方案

探索用 RAG 做文献检索和智能问答的技术方案

为轻量化的 AI 应用设计提供参考

实际意义：

降低 AI 对话系统的使用门槛和部署成本

保护用户隐私，数据都存在本地

提供文档解析和引用溯源的学术研究辅助功能

给个人开发者和小型团队提供一个能用的方案

1.2 国内研究状况及分析

国内商业产品：

百度文心一言：百度推出的大语言模型。它在中文场景下表现不错，能做多轮对话、代码生成、图片理解。

阿里通义千问：阿里巴巴通义实验室研发的模型系列。包括 Qwen-7B、Qwen-14B、Qwen-72B 等多个版本，部分模型已经开源^[6]。

腾讯混元：腾讯推出的大模型，主要做多模态理解和生成。

科大讯飞星火：科大讯飞推出的认知大模型，在语音交互和专业领域比较有优势。

国内开源生态现状

开源模型：ChatGLM，Qwen，Baichuan，MOSS 等

开源框架：LangChain-Chatchat，Dify，FastChat 等

本地部署方案：Ollama，LM Studio 等^[7,8,9]

现在的 AI 对话系统主要有四个问题。第一，大部分需要联网才能用，不支持离线运行。第二，用户数据要上传到云端处理，有泄露风险。第三，缺少针对学术研究的功能，比如文档解析、知识库检索、引用溯源这些。第四，界面和交互做得不太好，很多开源项目不太重视 UI 设计。

1.3 研究目标与内容

本文的目标是做一个能在本地跑起来的 AI 学术对话系统，不用连云端，也不用花钱买 API 额度。它要能读文档、检索知识、做智能对话，具体来说要满足下面几点：

- 1) 支持本地独立运行，不需要云服务
- 2) 支持多种开源大语言模型，方便切换
- 3) 支持上传 PDF/DOCX 文件，自动分段后存入知识库
- 4) 界面简洁、操作流畅、响应快速

1.4 主要技术路线

1.4.1 技术选型

本系统用到的核心技术如下：

表 1.1 技术栈

层级	技术	说明
前端构建工具	Vite	基于原生 ES Modules 的前端构建工具
前端语言	TypeScript	静态类型检查
样式方案	Tailwind CSS 4	原子化 CSS 框架
动画库	Framer Motion	React 动画库
图标库	Lucide React	现代图标库
后端框架	FastAPI	高性能 Python Web 框架
模型服务	Ollama	本地大模型运行框架
ORM	SQLAlchemy	Python ORM 框架

前端用 Vite + React 开发。Vite 是基于原生 ES modules 的构建工具，冷启动和热更新都很快，比传统的打包工具性能好很多。开发语言用 TypeScript，它会在写代码的时候检查类型，避免很多低级错误，项目大了也好维护。样式方面用 Tailwind CSS，不用写自定义 CSS 类，能快速做出统一的界面。状态管理用 TanStack React Query，用来处理服务端数据的缓存、自动刷新和乐观更新^[10,11]。

后端用 FastAPI 开发，它基于 Python，性能好，支持异步，还能自动生成接口文档。模型方面用 Ollama，可以把开源大模型跑在本地，不用联网。用户认证用 JWT (JSON Web Token)，不需要在服务端存 session。数据库用 SQLite，不用单独部署，一个文件

就搞定，很适合轻量级应用。版本控制用 `Git`^[12]。

1.4.2 开发环境

操作系统: Windows 11 Node.js: v20.x

Python: 3.12

代码编辑器: pycharm

2 需求分析

RETALK 系统要完成四个核心功能：AI 对话、文档管理、会话管理、引用溯源

2.1 功能需求

2.1.1 AI 对话功能

AI 对话功能主要是连续聊天、输出回复、管理消息、检索文档知识，具体如表 2.1 所示。

表 2.1 AI 对话功能需求

名称	详细描述
多轮对话支持	支持连续多轮对话交互，保持对话上下文逻辑连贯
非流式响应	调用 Ollama 同步接口获取完整回复后一次性展示，等待期间显示 Thinking 动画
消息管理	支持对话消息的复制、删除等常规管理操作
RAG 文档检索	基于上传文档的知识库检索，AI 回复附带 [1][2] 格式引用标注

2.1.2 文档管理功能

表 2.2 文档管理功能需求

名称	详细描述
文档上传	支持 PDF、DOCX 格式文件上传，限制大小不超过 10MB
文档解析	上传后自动提取文本并使用递归分块算法切分为约 500 tokens 的片段
文档列表	展示已上传文档的标题、大小、解析状态和分段数
文档删除	支持删除文档，同时清理磁盘文件和数据库记录

2.1.3 会话管理功能

会话管理功能主要是展示历史记录、切换会话、自动保存，具体如表 2.3 所示。

表 2.3 会话管理功能需求

名称	详细描述
会话列表	展示历史对话会话记录，便于快速查阅
会话操作	支持会话的新建与删除操作
会话切换	支持在不同会话间快速跳转切换
自动保存	对话内容通过 API 自动保存至服务端数据库

2.1.4 引用溯源功能

表 2.4 引用溯源功能需求

名称	详细描述
响应式布局	界面适配电脑屏幕尺寸
加载动画	提供加载状态动画，提升操作等待阶段的视觉体验
Toast 提示	操作执行后提供实时轻量提示，反馈操作结果

2.2 性能需求

2.2.1 服务端性能需求

系统要接收大量文本请求并处理。因为用户发消息后希望马上看到回复，所以响应速度很重要。要求：大模型推理的响应时间不超过 30 秒，API 接口响应不超过 100ms（不含模型推理）。

2.3 其他需求

2.3.1 健壮性需求

健壮性说的是系统遇到异常时的处理能力。为了让 RETALK 稳定运行，需要处理各种

异常情况，并及时告诉用户。具体来说，系统要能处理三类异常：网络异常（比如 API 请求失败）、模型异常（比如模型没加载、推理出错）、数据异常（比如无效输入、文件格式不对）。出错的时候，要给用户清晰的错误信息和解决建议。另外，异常情况出现时不能丢数据，要有自动保存和数据恢复功能。如果模型服务挂了，系统要能友好地提示用户，保证基本的可用性。

2.3.2 易用性需求

易用性直接影响用户愿不愿意继续用这个系统。一个做得好的 AI 对话工具，界面得一眼能看懂，操作别超过两三步，反馈要及时。具体来说就是：第一次打开不用看说明就知道怎么用；发消息、传文档这些高频操作，点两下就搞定；键盘快捷键得有，Enter 发送、Shift+Enter 换行是基本配置；等待的时候要有加载动画，别让人干等着不知道系统在干嘛；操作完了用 Toast 弹一下告诉用户成功还是失败；第一次进来的时候给一段引导文字，让人知道这个系统能干什么。

2.3.3 可扩展性需求

软件要长期迭代和更新，所以系统得方便扩展。具体来说：各功能模块之间要解耦，改一个不影响别的；预留插件接口，方便以后加新功能，比如接入向量数据库、支持多模态输入；核心参数要能配置，不用改代码就能调；代码要遵守统一的规范，方便以后的人接手。

2.4 功能模型

2.4.1 用例图

系统主要包含四大功能模块：对话、文档管理、会话管理、引用溯源。底层 AI 推理由

Ollama 本地大模型服务提供，如图 2.1 所示。

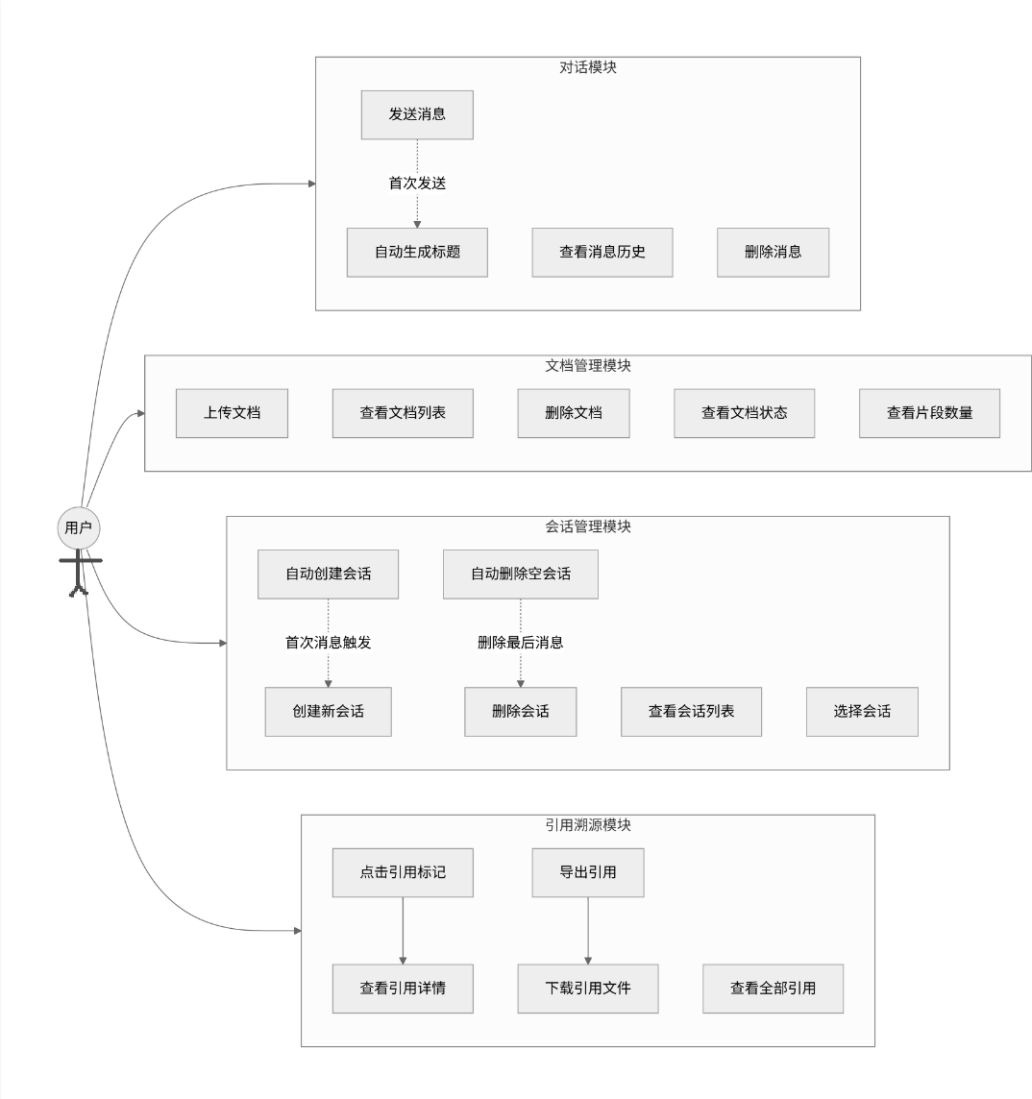


图 2.1 用例图

2.4.2 用例描述

表 2.5 到表 2.11 是各用例的详细描述，包括编号、前后置条件、基本流程和备选流程^[13]。

表 2.5 AI 对话用例描述表

项目	内容
用例名称	AI 对话
参与者	用户

前置条件	用户已登录，系统已启动，Ollama 模型服务正常运行
后置条件	用户获得 AI 回复（含引用标注），对话记录已保存至数据库
基本流程	1. 用户输入消息 2. 前端发送 POST 请求到 /api/v1/chat/completions 3. 后端执行 RAG 检索获取相关文档片段 4. 后端构建 Prompt 调用 Ollama 模型 5. 获取完整回复后返回前端 6. 前端展示 AI 回复和引用 标注 7. 保存对话记录到数据库
备选流程	模型服务异常：显示错误提示，建议用户重试

表 2.6 新建对话用例描述

项目	内容
用例名称	上传文档
参与者	用户
前置条件	用户已登录
后置条件	文档上传成功，文本已解析并分块存入数据库
基本流程	1. 用户在文档管理区域点击上传按钮，或者直接把文件拖进去 2. 前端 把文件包成 FormData，发到 POST /documents/upload 3. 后端先看文件 类型对不对（只收 PDF 和 DOCX）、大小有没有超（上限 10MB） 4. 校 验通过后，给文件起一个不会重复的名字，存到服务器的 uploads 文件 夹 5. 然后调解析服务：PDF 用 pypdf 提取文本，DOCX 用 python- docx，提完之后递归分块 6. 分好的文本片段写进数据库，文档状态更新 成“已解析” 7. 前端刷新文档列表，新上传的文档出现在最上面，显示分 段数量
备选流程	1. 文件类型不支持：提示仅支持 PDF、DOCX 2. 文件过大：提示文件不 能超过 10MB 3. 解析失败：文档状态标记为 FAILED

表 2.7 切换会话用例描述

项目	内容
用例名称	切换会话

参与者	用户
前置条件	存在历史会话
后置条件	成功切换到目标会话，显示历史消息
基本流程	1. 用户看一眼左边侧边栏里的会话列表 2. 直接点想切过去的那条会话 3. 前端调用 GET /conversations/{id} 拉取该会话的消息记录 4. 把当前聊天区域的内容替换成目标会话的历史消息 5. 侧边栏里被选中的条目高亮，之前的取消高亮

表 2.8 删除会话用例描述

项目	内容
用例名称	删除会话
参与者	用户
前置条件	系统已启动，存在可操作的历史会话
后置条件	目标会话被成功删除，会话列表更新，不再显示该会话
基本流程	1. 在侧边栏的会话列表中找到要删除的条目 2. 鼠标移上去后，条目右侧会出现一个删除小图标 3. 点这个图标，屏幕中央弹出一个确认框，问“确定删除？” 4. 点“确定”之后，前端调用 DELETE /conversations/{id} 接口 5. 后端把这条会话连同它下面的消息记录一起从数据库里删掉 6. 前端收到成功响应后，从列表里移除这条会话，如果删的是当前正在看的会话，就自动跳到列表里的下一条
备选流程	1. 用户取消删除：关闭对话框，不执行任何操作 2. 会话不存在：提示“会话已失效”，刷新列表

表 2.9 新建会话用例描述

项目	内容
用例名称	新建会话
参与者	用户

前置条件	系统已启动并正常连接模型服务
后置条件	创建新的空会话，界面切换至全新对话状态
基本流程	1. 侧边栏顶部有一个“新建会话”的按钮，用户点一下 2. 前端发 POST /conversations 请求，后端在数据库里建一条新的会话记录，返回会话 ID 3. 前端拿到 ID 后把右边的聊天区域清空，标题显示“新对话” 4. 光标自动落在输入框里，用户可以直接打字开始聊 5. 左边的会话列表同步刷新，新建的这条排在最上面
备选流程	1. 模型服务异常：提示无法新建会话，请先恢复模型连接

表 2.10 引用导出用例描述

项目	内容
用例名称	引用导出
参与者	用户
前置条件	AI 回复中包含引用标注
后置条件	引用内容以 BibTeX 或 GB/T 7714 格式导出
基本流程	1. 用户查看 AI 回复中的引用 2. 点击导出按钮 3. 选择导出格式 4. 系统调用 GET /citations/export 接口 5. 返回格式化的引用文本
备选流程	消息无引用：提示当前消息没有引用信息

表 2.11 操作结果提示用例描述

项目	内容
用例名称	操作结果提示
参与者	用户

项目	内容
前置条件	用户执行关键操作（新建 / 删除 / 保存）
后置条件	用户通过 Toast / 弹窗知晓操作成败
基本流程	1. 用户做完一个动作（比如删掉某个会话、上传一份文档） 2. 前端把请求发给后端，等后端返回结果 3. 拿到结果后，如果成功就显示绿色的 Toast 提示，比如“删除成功”；失败就显示红色的，比如“上传失败，文件格式不对” 4. 这个提示在屏幕右上角停留大概 3 秒后自动消失，用户也可以手动点关掉
备选流程	1. 异常操作：显示 “操作失败，请检查网络或重试”

实体关系

User（用户）和 Conversation（会话）：一个用户可以有多个会话

User（用户）和 Document（文档）：一个用户可以上传多个文档

Conversation（会话）和 Message（消息）：一个会话包含多条消息

Document（文档）和 Chunk（文本片段）：一个文档被切分成多个文本片段

2.5 静态模型

根据面向对象的思想，软件里的对象对应现实世界里的概念。所以，梳理领域模型是需求分析的重要一步^[14]。本系统的领域模型包含 5 个核心数据类：User（用户）、Document（文档）、Conversation（会话）、Message（消息）和 Chunk（文本片

段)，如图 2.2 所示。

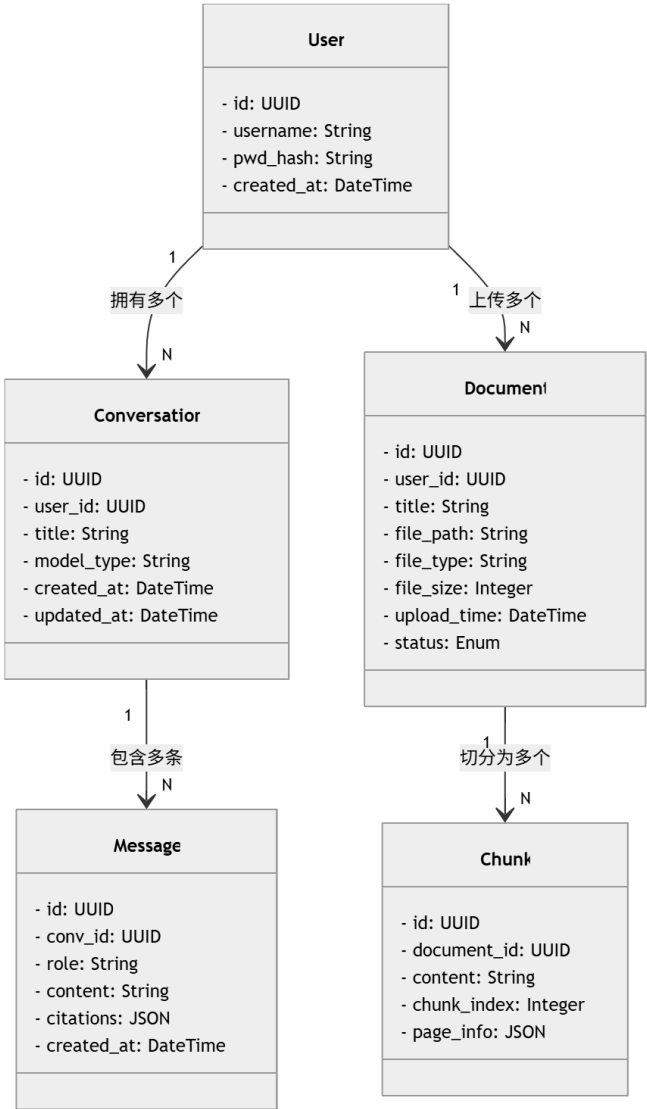


图 2.2 领域类图

3 概要设计

3.1 系统体系架构设计

3.1.1 整体架构

RETALK 系统采用前后端分离架构，整体分三层：用户层、应用层和模型层。用户层是前端应用，用 React + TypeScript + Tailwind CSS 开发，负责展示界面和接收用户操作。应用层是后端服务，用 Python + Uvicorn + FastAPI 搭建，负责处理请求和业务逻辑。模型层用 Ollama 跑 llama3.2 等大语言模型，负责 AI 推理。用户层和应用层之间用 HTTP/REST API 通信，应用层和模型层之间用本地 API 调用，如图 3.1 所示^[10, 11, 12]。

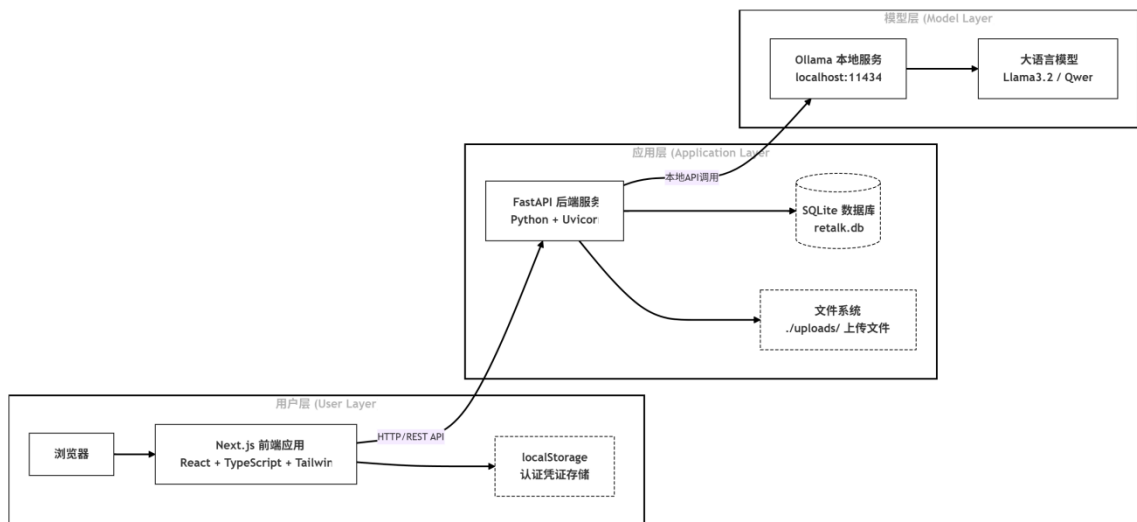


图 3.1 系统构件图

3.1.2 后台服务端架构设计

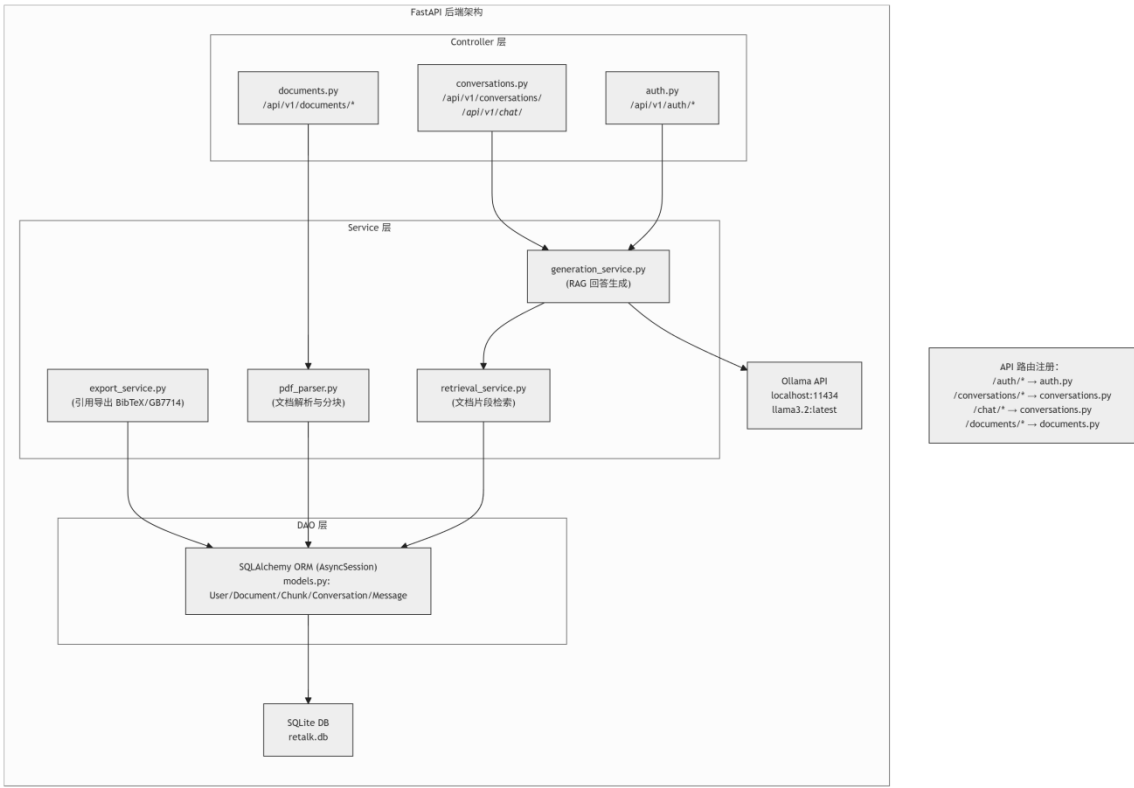


图 3.2 后端架构设计

后端用 FastAPI 开发，整体分为三层：Controller、Service 和 DAO。同时接了 Ollama 做模型推理，用 SQLite 做数据存储，如图 3.2 所示。Controller 层负责接收前端请求，并转给对应的业务逻辑。Service 层是核心，处理文档解析、RAG 检索、对话管理等业务。DAO 层负责读写 SQLite 数据库，保存对话记录、文档信息和消息。Ollama 交互模块在 Service 层里，负责调大模型拿到回复^[15]。

3.1.3 前端架构设计

前端用 Vite + React 开发，整体分为四层：页面路由层、组件层、工具层和静态资源层，如图 3.3 所示。页面路由层用 React Router 管理页面跳转和布局。组件层封装可复用的 UI 组件，比如消息气泡、输入框、上传区。工具层提供一些通用函数，比如时间格式化、文件大小格式化。静态资源层放图片和图标。React Query 负责数据缓存和自动刷新，和组件化开发配合，让页面加载快、交互流畅。

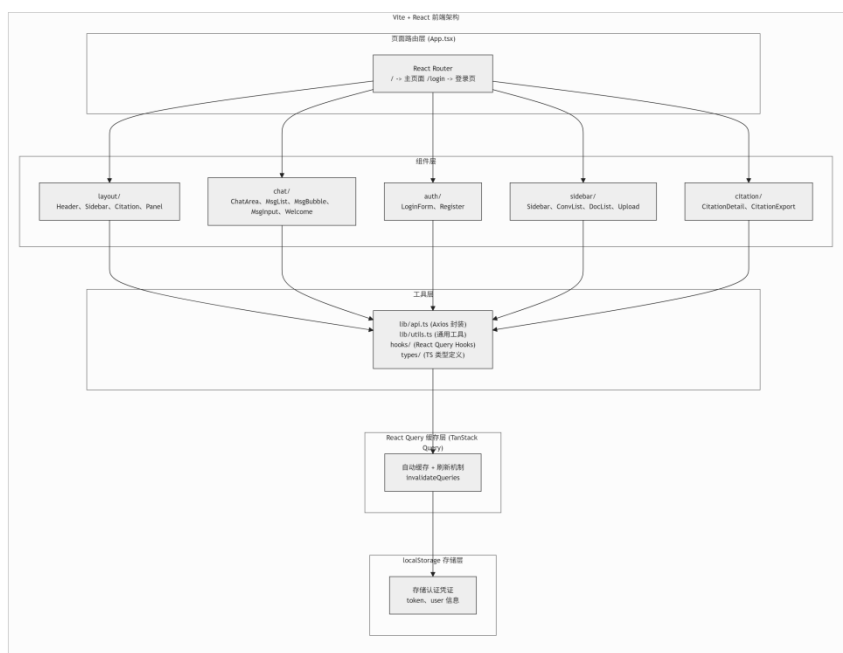


图 3.3 前端架构设计

3.2 数据设计

RETALK 系统的数据存储分三种：用户凭证用 JWT 存到浏览器的 localStorage；业务数据（用户、会话、消息、文档、文本片段）存在 SQLite 数据库；文件本身存在服务器本地的文件系统。数据流从用户输入到前端展示走一个完整闭环，如图 3.4 所示。

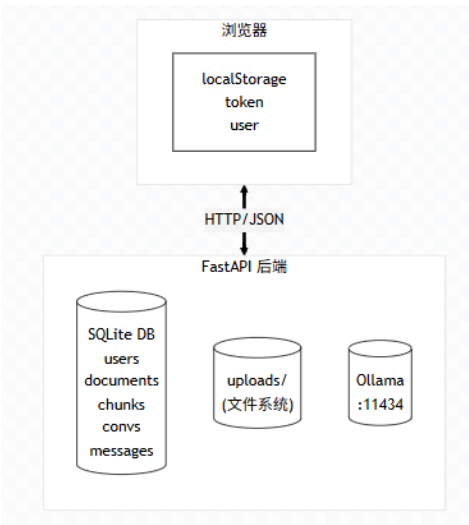


图 3.4 数据架构图

4 详细设计

4.1 系统层次结构

RETALK 系统分三层：表现层、业务层和数据层，如图 4.1 所示。表现层管页面和 UI 组件，负责展示界面和响应用户操作。业务层管核心逻辑，包括对话管理、文档管理和引用溯源。数据层管数据，包括 API 调用、JWT 认证和数据库访问。

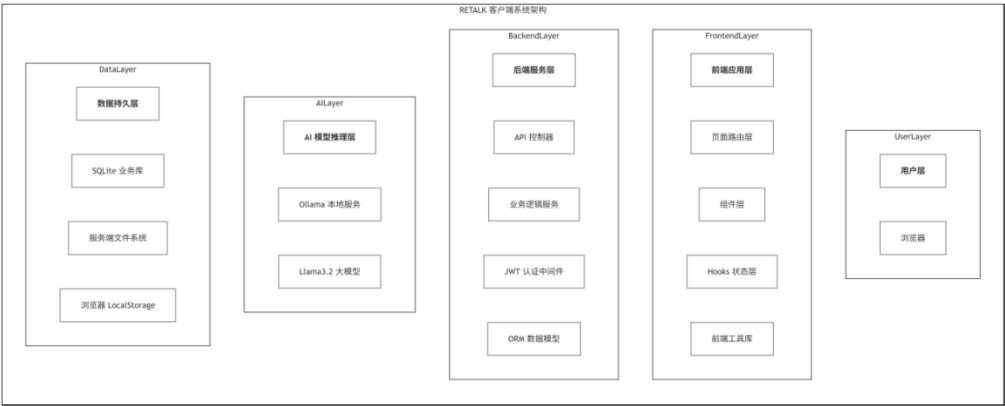


图 4.1 客户端系统架构图

4.2 消息发送模块 IPO 表

表 4.1 到表 4.5 是各功能模块的算法 IPO 表，包括功能编号、输入输出和相关数据 [16]。

表 4.1 消息发送模块 IPO 表

项目	内容
模块名称	消息发送模块
输入	1. 用户输入内容 (content: string)
(Input)	2. 当前对话 ID (conversationId: string, 可选)

项目	内容
处理 (Process)	1. 检查是否正在发送（防重复提交） 2. 设置 <code>isSending=true</code>，显示 Thinking 动画 3. 调用 <code>conversationApi.sendMessage</code> 发送 POST 请求到 <code>/api/v1/chat/completions</code> 4. 请求体包含 <code>message</code> 和 <code>conversation_id</code> 5. 等待后端完整响应（非流式） 6. 若为首条消息且无对话 ID，通知父组件创建新对话 7. 调用 <code>queryClient.invalidateQueries</code> 刷新消息和对话列表缓存
输出 (Output)	1. AI 回复内容 (<code>answer: string</code>) 2. 引用列表 (<code>citations: CitationResponse[]</code>) 3. 对话 ID (<code>conversation_id: string</code>)

表 4.2 文档上传模块 IPO 表

项目	内容
模块名称	文档上传模块
输入 (Input)	1. 用户上传的文件 (<code>file: UploadFile</code>) 2. 当前登录用户信息 (<code>current_user: User</code>)
处理 (Process)	1. 校验文件扩展名（仅允许 <code>.pdf</code>、<code>.docx</code>） 2. 读取文件内容并校验大小 3. 生成 UUID 唯一文件名，保存到 <code>uploads</code> 目录 4. 创建 <code>Document</code> 数据库记录（状态为 <code>UPLOADED</code>） 5. 调用 <code>parse_document</code> 解析文档文本 6. 对每页文本执行 <code>recursive_chunk</code> 递归分块 7. 将分块存入 <code>Chunk</code> 表，更新文档状态为 <code>PARSED</code> 8. 解析失败时标记状态为 <code>FAILED</code>
输出 (Output)	1. <code>DocumentResponse</code> (<code>id</code>、<code>title</code>、<code>file_type</code>、<code>file_size</code>、<code>status</code>) 2. 解析后的分块数量 (<code>chunk_count</code>)

表 4.2 会话管理模块 IPO 表

项目	内容
模块名称	会话管理模块
输入 (Input)	1. 操作类型 (新建 / 删除 / 切换) 2. 会话 ID (删除和切换时必填)
处理 (Process)	1. 新建: POST /conversations 创建空白对话, 返回对话 ID 2. 删除: DELETE /conversations/{id} 删除对话及关联消息 3. 切换: GET /conversations/{id} 加载目标对话的消息列表 4. 通过 React Query 的 invalidateQueries 自动刷新缓存 5. 对话列表按更新时间倒序, 前端按日期分组显示
输出 (Output)	1. 更新后的会话列表 2. 当前对话的消息数据

表 4.3 聊天接口 IPO 表

项目	内容
模块名称	聊天 API 接口 (POST /api/v1/chat/completions)
输入 (Input)	1. 用户消息文本 (message: str) 2. 对话 ID (conversation_id: Optional[str]) 3. 模型类型 (model_type: str, 默认 "ollama")

项目	内容
处理 (Process)	1. JWT 身份验证，获取当前用户 2. 调用 <code>generate_answer_with_citations</code> 执行 RAG 流程 3. 从 Chunk 表检索最多 5 条文档片段 4. 将片段编号后拼接为上下文，与用户问题组装成 Prompt 5. 使用服务端硬编码的 System Prompt 6. 调用 <code>ollama.chat</code> 同步获取完整回复 7. 用正则提取回复中的 [n] 引用编号 8. 若无对话 ID，自动创建 Conversation 9. 保存 user 和 assistant 两条 Message 10. 首轮对话时截取消息前 50 字符作为标题
输出 (Output)	1. AI 回复内容 (<code>answer: str</code>) 2. 引用列表 (<code>citations: List[CitationResponse]</code>) 3. 对话 ID (<code>conversation_id: str</code>)

表 4.5 引用导出接口 IPO 表

项目	内容
模块名称	引用导出接口 (GET <code>/api/v1/citations/export</code>)
输入 (Input)	1. 消息 ID (<code>message_id: str</code>) 2. 导出格式 (<code>format: str</code>, 支持 <code>bibtex</code> 或 <code>gb7714</code>)
处理 (Process)	1. 根据 <code>message_id</code> 查询 Message 记录 2. 提取 <code>citations</code> JSON 字段中的引用列表 3. 调用 <code>export_citations</code> 服务 4. BibTeX: 生成 @article 条目 5. GB/T 7714: 按国标格式输出
输出 (Output)	格式化后的引用文本字符串

4.3 数据库设计

数据库设计就是设计数据库本身^[17]。系统的基本数据是五类：用户、文档、会话、消息、文本片段。围绕这些对象设计了五张表，如表 4.6 到表 4.10 所示。

用户表（users）保存用户的账户信息，包括用户名和密码哈希，如表 4.6 所示；

表 4.6 用户表（users）

字段名	类型	约束	说明
id	VARCHAR(36)	PRIMARY KEY, DEFAULT uuid4()	主键 (UUID)
username	VARCHAR(20)	UNIQUE, NOT NULL, INDEX	用户名
password_hash	VARCHAR(255)	NOT NULL	密码哈希 (bcrypt)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	创建时间

会话表（conversations）保存用户的对话会话，关联用户表，如表 4.7 所示

表 4.7 会话表（conversations）

字段名	类型	约束	说明
id	VARCHAR(36)	PRIMARY KEY, DEFAULT uuid4()	主键 (UUID)
user_id	VARCHAR(36)	FOREIGN KEY -> users.id, NOT NULL	所属用户

字段名	类型	约束	说明
title	VARCHAR(200)	NOT NULL	会话标题

续表 4.7 会话表 (conversations)

字段名	类型	约束	说明
model_type	VARCHAR(50)	DEFAULT 'ollama'	模型类型
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP, ON UPDATE	更新时间

文档表 (documents) 保存用户上传的文档信息，包括标题、路径、类型、大小、解析状态，关联用户表，如表 4.8 所示；

表 4.8 文档表 (documents)

字段名	类型	约束	说明
id	VARCHAR(36)	PRIMARY KEY, DEFAULT uuid4()	主键 (UUID)
user_id	VARCHAR(36)	FOREIGN KEY -> users.id, NOT NULL	所属用户
title	VARCHAR(500)	NOT NULL	文档标题 / 文件名
file_path	VARCHAR(500)	NOT NULL	服务端存储路径
file_type	VARCHAR(20)	NOT NULL	文件类型 (PDF / DOCX)
file_size	INTEGER	NOT NULL	文件大小 (字节)

字段名	类型	约束	说明
upload_time	DATETIME	DEFAULT CURRENT_TIMESTAMP	上传时间
status	ENUM	DEFAULT 'uploaded'	解析状态 (uploaded/parsed/failed)

消息表（`messages`）保存对话中的消息内容、角色类型和引用信息，关联会话表，如表 4.9 所示。

表 4.9 消息表（`messages`）

字段名	类型	约束	说明
id	String	PRIMARY KEY	消息唯一标识（UUID 格式）
conversation_id	String	FOREIGN KEY->conversations.id NOT NULL	关联的会话 ID
role	String(20)	NOT NULL	消息角色类型：“user” （用户提问）或 “assistant”（AI 回 复）
content	Text	NOT NULL	消息内容正文
citations	JSON	DEFAULT []	引用信息列表（JSON 数组，存储 AI 引用 的文档片段）
created_at	DateTime	DEFAULT UTC NOW	消息创建时间戳

文本片段表（**chunks**）保存文档切分后的文本内容、片段索引和页码信息，关联文档表，如表 4.10 所示。

表 4.10 文本片段表（**chunks**）

字段名	类型	约束	说明
id	VARCHAR(36)	PRIMARY KEY, DEFAULT uuid4()	主键（UUID）
conversation_id	VARCHAR(36)	FOREIGN KEY -> conversations.id, NOT NULL	所属会话
role	VARCHAR(20)	NOT NULL	角色（user / assistant）
content	TEXT	NOT NULL	消息内容
citations	JSON	DEFAULT []	引用列表（AI 回复 附带）
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	创建时间

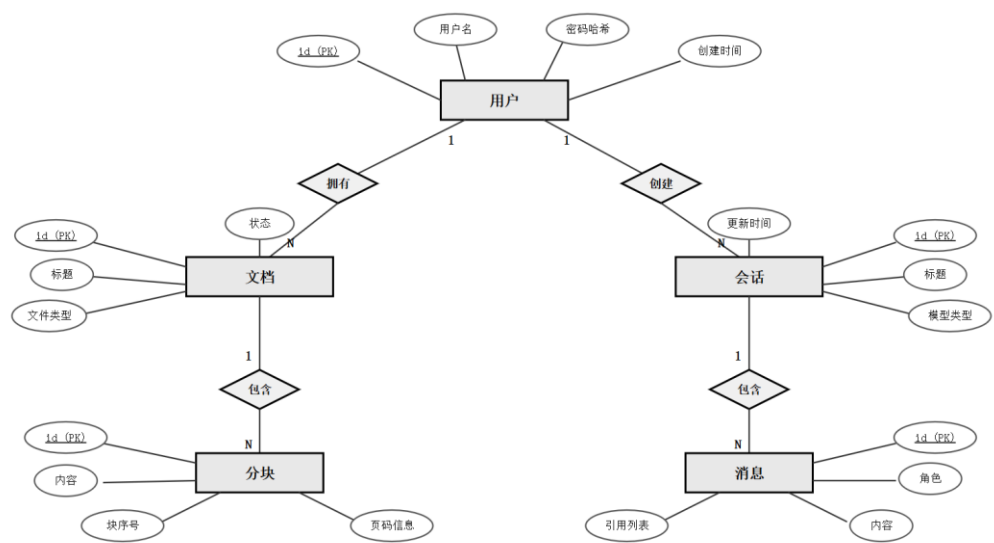


图 4.2 实体关系图 (ER 图)

5 系统实现

5.1 文档管理功能实现

要让 AI 在回答问题时能够引用文档内容，光把整篇文档丢给模型是不行的——一方面太长放不下，另一方面模型容易"迷路"找不到关键信息。所以系统做了一套文档上传、解析和知识库管理的流程。大致是这样：用户把 PDF 或者 DOCX 文件拖进上传区域，后端收到文件后先把文本内容提取出来，接着用一个递归分块的算法，沿着段落、句子这些自然边界把文本切成一段一段的，每段控制在 500 个 token 上下，然后写进 SQLite 数据库。解析完成后，用户在聊天框里提问的时候，系统就能从这些片段里找到最相关的几条来辅助回答。如果上传过程中出了问题，比如文件格式不对或者解析报错，系统会把这份文档标记成"失败"状态并告诉用户。

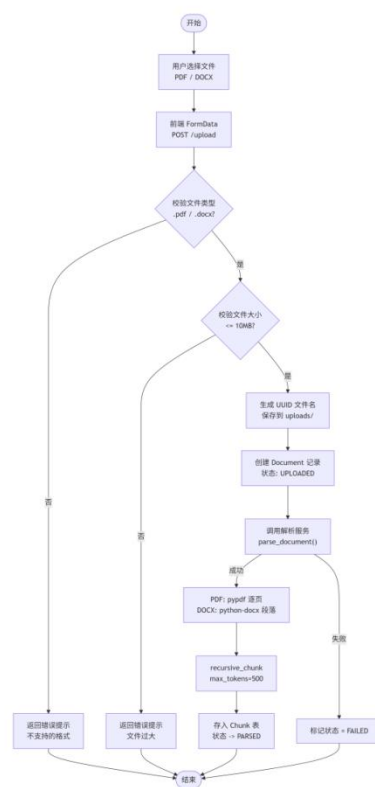


图 5.1 文档管理功能流程图

5.1.1 界面设计

知识库面板在侧边栏的知识库标签页里。页面上方是文件上传区域，支持点击上传

和拖拽上传两种方式，上传时会显示进度。下方是文档列表，显示每个文档的标题、大小、解析状态和分段数。每个文档旁边有个删除按钮，方便管理。界面设计如图 5.2 所示。



图 5.2 文档管理功能界面

5.1.2 功能实现

本文档上传与管理功能的核心是文件处理、文本提取和分块存储。下面介绍关键实现。

- 文档上传与解析实现

文件上传的实现分前后端两部分。前端这边，用户选完文件后，代码把文件塞进 FormData 对象，通过 axios 发到 POST /documents/upload。后端收到请求，第一步是检查：文件扩展名必须是 .pdf 或 .docx，大小不能超过 10MB，不满足就直接返回 400 错误。检查通过后，用 uuid 拼出一个不重复的文件名，写到 uploads 目录下。接下来调解析服务——PDF 走 pypdf 的 PdfReader 逐页提取文字，DOCX 走 python-docx 逐段拼接。拿到完整文本后丢给 recursive_chunk() 做分块，分好的片段连同文档 ID 一起批量插入 chunks 表。最后把文档状态从 PENDING 改成 PARSED，前端轮询

到状态变化后刷新列表。

- 创建自定义分块

`recursive_chunk()` 是分块的核心函数。它先用字符数除以 4 来粗略估算 token 数量，如果一段文本不超过上限（默认 500 token）就直接返回。超了的话，就找中间位置的段落分隔符（优先双换行，其次单换行，再其次是句号），从那里一刀切成两半，然后对每一半递归地再切，直到每段都在限制以内。

如果 token 数量没超过限制，就直接返回这段文本；如果超过了，先按段落（两个换行符）分开，然后一段段往当前块里加

加着加着如果超出限制了，就把当前块存起来，再开一个新块；如果段落分不开，就再按句号、问号等分隔符进一步切。

`chunk_index`：每个文本片段有一个递增的编号，用来在文档内唯一标识，方便后面检索和引用

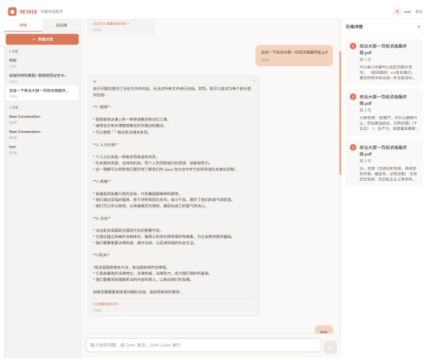
- RAG 检索增强实现^[15]

用户发消息的时候，系统从数据库取出最近的相关文本片段（取 5 条），把它们和用户的消息一起组装成 Prompt，然后调 Ollama API。Ollama 回复的时候，会用 [1][2][3] 这样的格式标注引用编号。前端用正则表达式把这些编号解析出来，变成可点击的引用链接。用户点击链接就能看到对应的文本片段内容。

5.2 对话功能实现

5.2.1 界面设计

对话功能是系统的核心功能，页面采用聊天布局。流程是：用户进入对话页面，输入内容并发送，系统先执行 RAG 检索（从知识库取相关文本片段），然后组装 Prompt 调 Ollama，拿到完整回复后在聊天界面展示出来。如果请求失败就提示错误。流程如图 5.3 所示。



(a) 对话页面



(b) 侧边栏展示



(c) 对话结果展示

图 5.3 对话界面设计

5.2.2 功能实现

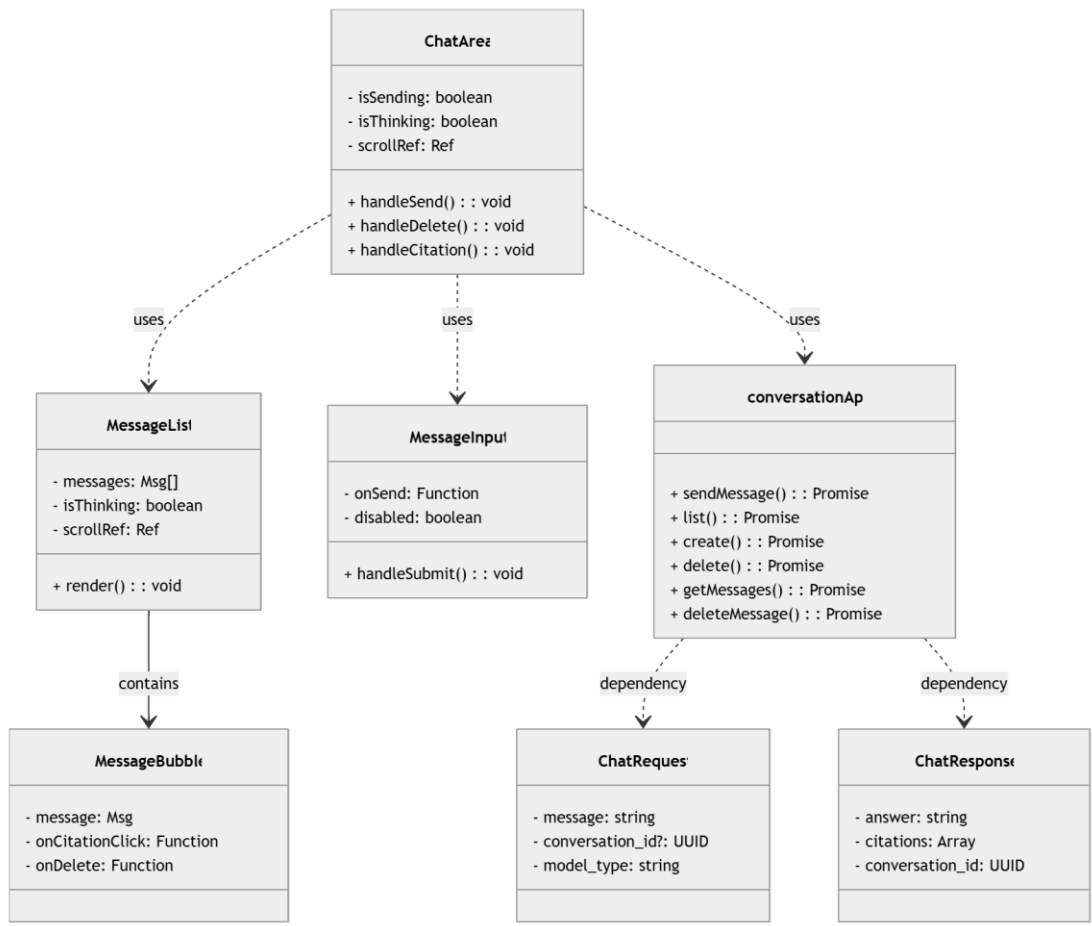


图 5.4 对话功能类图

ChatArea 是对话页面的核心组件，管理对话状态和用户交互。Conversation（对话）和 Message（消息）定义数据格式。localStorage 存用户凭证。conversationApi 封装和 AI 服务的接口。ChatRequest 规范 API 请求参数。这几个模块分工合作，完成对话的发起、交互和存储。

● 前端消息发送流程

用户发消息时，前端先显示加载状态（Thinking 动画）。然后组装请求体（包含 conversation_id 和 message），向后端 /api/v1/chat/completions 发 POST 请求。请求成

功的话，就把 AI 回复和引用信息存到消息列表里。失败的话就显示错误提示。

```
// 前端消息发送流程 (ChatArea.tsx)

const handleSend = async (content: string) => {
  if (isSending) return;
  setIsSending(true);
  setIsThinking(true);

  try {
    const response = await onSendMessage?.(content);
    if (!response) return;

    // 自动创建对话
    if (!conversationId && onConversationCreated) {
      onConversationCreated(response.conversation_id);
    }

    // 刷新消息缓存
    queryClient.invalidateQueries({
      queryKey: ['messages', response.conversation_id]
    });
    queryClient.invalidateQueries({
      queryKey: ['conversations']
    });
  } catch (error) {
    console.error('发送消息失败:', error);
  } finally {
    setIsSending(false);
    setIsThinking(false);
  }
};
```

● 后端聊天接口实现

后端 `/api/v1/chat/completions` 接口收到消息后，调用 `generate_answer_with_citations` 函数执行 RAG 流程。先从数据库取相关文本片段，组装 Prompt 和系统提示词，然后用 `ollama.chat` 同步调用 Ollama 拿到回复，再用正则表达式从回复中提取引用标注

（[1][2][3]格式），映射成对应的文本片段数据。如果 Ollama 服务挂了，就返回错误信息。成功的话就把用户消息和 AI 回复存到数据库，返回结果。

```
# 后端聊天接口实现 (conversations.py)
@router_chat.post("/completions")
async def chat_completion(
    request: ChatRequest,
    db: AsyncSession = Depends(get_db),
    current_user: User = Depends(verify_jwt_token)
):
    try:
        answer, citations = await generate_answer_with_citations(
            message=request.message,
            db=db,
        )
    except Exception as e:
        answer = f"抱歉，生成回答时出错：{str(e)}"
        citations = []

    # 保存用户消息和 AI 回复
    user_message = Message(conversation_id=conversation_id,
                           role="user", content=request.message)
    assistant_message = Message(conversation_id=conversation_id,
                                role="assistant", content=answer,
                                citations=citations)

    db.add(user_message)
    db.add(assistant_message)
    await db.commit()

    return ChatResponse(answer=answer, citations=citations,
                        conversation_id=conversation_id)
```

5.3 用户模块功能实现

用户要先登录才能用系统的功能，比如上传文档、对话聊天等。

5.3.1 界面设计

登录和注册页面居中显示。用户可以切换登录/注册标签页，输入用户名和密码提交。输入不对的话会提示修改。登录成功后，后端返回 JWT Token，前端存到 localStorage，然后跳转到主界面。流程如图 5.5 所示。

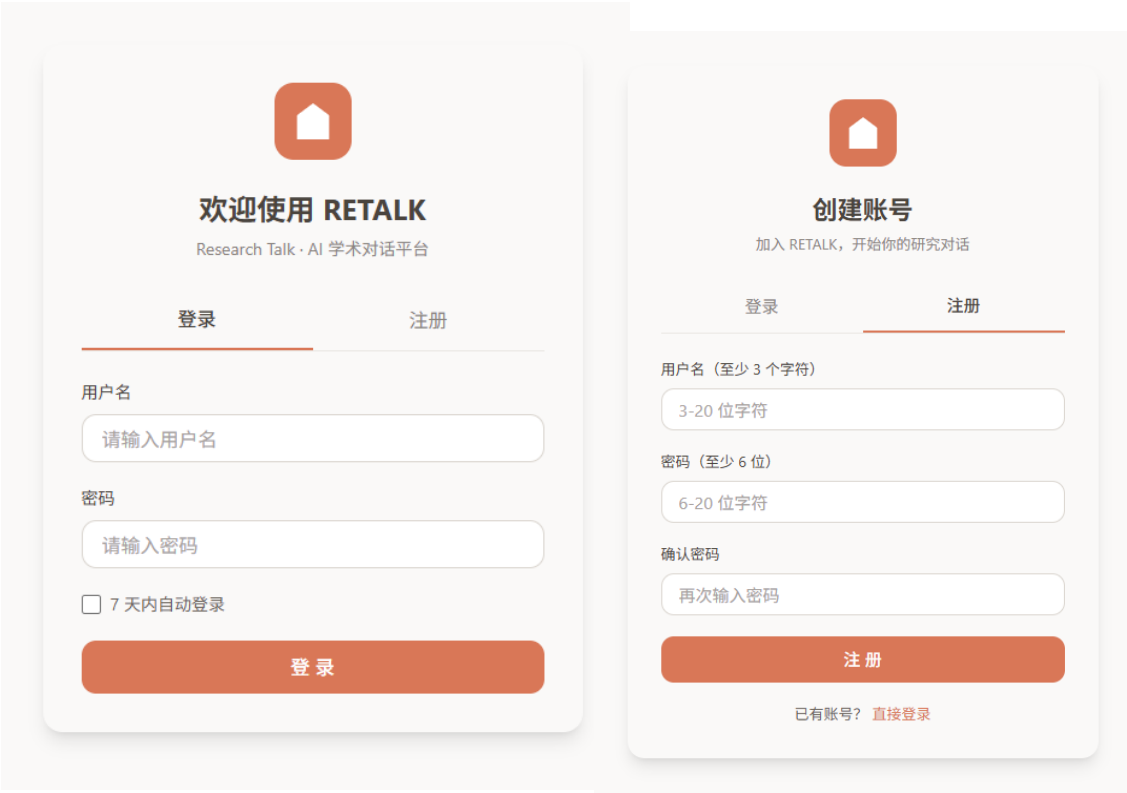


图 5.5 用户模块界面

5.3.2 功能实现

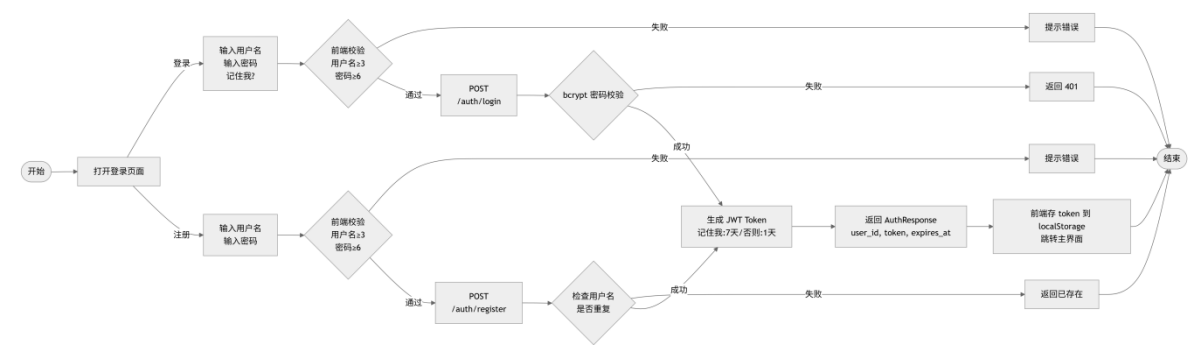


图 5.6 用户登录/注册流程图

登录注册功能在前端和后端协同完成。前端检查用户名（至少 3 位）和密码（至少 6 位）的格式。后端收到请求后，用 bcrypt 做密码哈希校验。校验通过后，用 JWT 生成访问令牌，返回给前端。前端把 Token 存到 localStorage，后面的每个 API 请求都会带上这个 Token。后端通过 verify_jwt_token 验证 Token 是否有效，确保接口安全。

```
// 用户登录实现 (LoginForm.tsx)
export default function LoginForm({ onSwitchToRegister, onSuccess }) {
  const [username, setUsername] = useState("")
  const [password, setPassword] = useState("")
  const [rememberMe, setRememberMe] = useState(false)
  const [error, setError] = useState("")

  const handleSubmit = async (e) => {
    e.preventDefault()
    setError("")

    if (!isValidUsername(username)) {
      setError('用户名至少 3 个字符')
      return
    }
    if (!isValidPassword(password)) {
      setError('密码至少 6 个字符')
      return
    }

    try {
      await loginMutation.mutateAsync({
        username, password, remember: rememberMe
      })
      onSuccess?.()
    } catch (err) {
      setError('登录失败，请检查用户名和密码')
    }
  }

  // ... 表单渲染代码
}
```


总结与展望

本文设计并实现了一个 AI 学术研究对话系统，完成了预期目标。但因为开发周期、硬件条件和技术储备的限制，还有一些不足：

1. 模型能力有限：开源小模型处理复杂任务不如商业大模型，高难度的学术对话场景下效果不够好。
2. 硬件要求较高：跑大模型需要一定配置的硬件，普通用户可能达不到。
3. 检索精度不够：目前直接用文本片段检索，没用向量相似度搜索，文档多了之后准确率会下降。
4. 移动端适配不足：在手机上的界面效果还需要优化。

接下来打算从几个方向继续改进。模型这块，想试试量化技术（比如 4-bit 量化）来降低显存占用，同时接入更多开源模型给用户更多选择。检索这块是当前最大的短板，后面准备引入 ChromaDB 这样的向量数据库，用语义相似度来找文本片段，而不是现在这种靠关键词匹配的土办法。功能上，多模态是一个值得探索的方向，比如让用户上传图片或者表格，模型也能理解和引用；另外知识库管理的界面也需要重新设计，现在太简陋了。体验上，移动端的适配还差得比较远，布局和交互都要重做；还想加一个主题切换的功能，至少支持深色模式。

参考文献

- [1]Vaswani A, Shazeer N, Parmar N, et al.Attention is all you need[C]//Advances in Neural Information Processing Systems.Cambridge:MIT Press,2017:5998-6008.
- [2]Brown T, Mann B, Ryder N, et al.Language models are few-shot learners[J].Advances in Neural Information Processing Systems,2020,33:1877-1901.
- [3]OpenAI.GPT-4 technical report[R/OL].arXiv preprint arXiv:2303.08774,2023.
- [4]李航.深度学习[M].北京:清华大学出版社,2019:1-50.
- [5]周志华.机器学习[M].北京:清华大学出版社,2016:1-40.
- [6]Bai J, Bai S, Chu Y, et al.Qwen technical report[R/OL].arXiv preprint arXiv:2309.16609,2023.
- [7]Touvron H, Martin L, Stone K, et al.Llama 2:open foundation and fine-tuned chat models[R/OL].arXiv preprint arXiv:2307.09288,2023.
- [8]Meta AI.Llama 3.2:revolutionizing edge AI and vision with open, customizable models[R/OL].arXiv preprint,2024.
- [9]Ollama Team.Ollama:run large language models locally[CP/OL].<https://github.com/ollama/ollama>,2023.
- [10]You E.Vite:next generation frontend tooling[CP/OL].<https://github.com/vitejs/vite>,2020.
- [11]Tailwind Labs.Tailwind CSS[CP/OL].<https://github.com/tailwindlabs/tailwindcss>,2017.
- [12]Ramírez S.FastAPI[CP/OL].<https://github.com/fastapi/fastapi>,2018.
- [13]张海藩,牟永敏.软件工程导论[M].6 版.北京:清华大学出版社,2013:60-90.
- [14]Larman C.Applying UML and patterns[M].3rd ed.Upper Saddle River:Prentice Hall,2004:100-150.
- [15]Lewis P, Perez E, Piktus A, et al.Retrieval-augmented generation for knowledge-intensive NLP tasks[C]//Advances in Neural Information Processing Systems.Cambridge:MIT Press,2020:9459-9474.
- [16]郑人杰,殷南京.实用软件工程[M].4 版.北京:清华大学出版社,2015:120-150.
- [17]王珊,萨师煊.数据库系统概论[M].5 版.北京:高等教育出版社,2014:150-200.

致谢

写到这里,我的毕业设计也结束了,大学生活即将进入新阶段。这段时间做这个 AI 对话系统,写毕业论文,让我学到了很多,也经历了很多事情,不过都一一克服,更深刻理解了习主席的话:“石可破也,而不可夺坚;丹可磨也,而不可夺赤。”未来和计划永远不会一帆风顺一成不变,只要迈过去都将成为我成长的养分。感谢身边老师、朋友和家人的帮助,在这里表达我的谢意。

首先感谢我的指导老师。从选题、需求分析、架构设计、功能实现,到论文的写作和修改,每一步都离不开老师的指导。老师在我遇到困难的时候总能给出中肯的建议,他严谨的态度和扎实的专业知识让我受益匪浅。

其次感谢大学期间教过我的所有老师。从基础的编程知识到 AI、软件开发的专业理论,老师们的课堂讲解和课后答疑帮我建立了知识体系。正因为有了这些基础,我才能独立完成这个系统的设计和开发。

最后感谢我的家人。这些年他们一直在物质和精神上支持我,在我最困难的时刻安慰我,帮助我走出心中的坎坷,让我能安心学习,专心做项目。这份支持是我前进的动力。

因为我的专业知识和开发经验还有不足,这个系统和论文可能还有不完善的地方,希望各位老师、同学和专家批评指正。这次毕业设计的经历让我认识到自己的不足,也让我明白学习是永无止境的。未来我会继续努力,不断提升自己。

前路漫漫,道阻且长。